

Relatório de Laboratório

Laboratório de Experimentação de Software — Modelo/Template de Relatório

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Mineração de Métricas DORA
Grupo (trio)	Enzo Casaes · Cauê Afonso · Leonardo Viana
Link do repositório / GitHub Projects	https://github.com/Enzocasaes/Laboratorio-medicao-experimentacao-de-software
Data de entrega	21/08/2026

1. Introdução

Projetos de código aberto possuem papel relevante no desenvolvimento de software, sendo utilizados como base para aplicações, ferramentas e bibliotecas na indústria e na academia. Embora a popularidade desses projetos possa ser facilmente observada pelo número de estrelas no GitHub, essa métrica, isoladamente, não permite compreender como os repositórios são desenvolvidos e mantidos. Aspectos como maturidade, participação de contribuidores externos, frequência de releases e atualizações e tratamento de issues podem apresentar comportamentos distintos mesmo entre projetos igualmente populares.

*Diante desse problema, este laboratório busca caracterizar empiricamente os 1.000 repositórios com maior número de estrelas no GitHub, utilizando dados coletados diretamente por meio da API GraphQL da plataforma. A análise é orientada pelas seguintes questões de pesquisa: **RQ01**: qual é a idade dos repositórios mais populares? **RQ02**: qual é a proporção de pull requests externas aceitas? **RQ03**: com que frequência esses projetos publicam releases? **RQ04**: há quanto tempo os repositórios foram atualizados pela última vez? **RQ05**: quais são as linguagens de programação predominantes entre os repositórios mais populares? **RQ06**: qual é a proporção de issues fechadas? **RQ07**: existe relação entre a popularidade das linguagens utilizadas e características como contribuição externa, frequência de releases e atualizações? **RQ08 (inovação do grupo)**: existe correlação entre a popularidade dos repositórios e as demais características investigadas, como contribuição externa, frequência de releases e de atualizações?*

*Antes da coleta dos dados, foram formuladas hipóteses informais para orientar a investigação. Para a **RQ01**, espera-se que os projetos sejam, em sua maioria, maduros e possuam vários anos de existência. Para a **RQ02**, espera-se uma participação significativa de desenvolvedores externos, refletida por um número elevado de pull requests aceitas. Para a **RQ03**, espera-se que projetos populares publiquem releases com relativa frequência. Para a **RQ04**, espera-se que os repositórios sejam atualizados regularmente e apresentem pouco tempo desde sua última modificação. Para a **RQ05**, espera-se predominância das linguagens mais utilizadas pela comunidade de desenvolvimento, segundo o índice de popularidade adotado no laboratório. Para a **RQ06**, espera-se uma proporção elevada de issues fechadas, indicando atividade de manutenção. Para a **RQ07**, espera-se que projetos escritos nas linguagens mais populares apresentem, em geral, maior contribuição externa, maior frequência de releases e atualizações mais frequentes. Como inovação do grupo, para a **RQ08** espera-se correlação positiva fraca a moderada entre popularidade e contribuição externa, e correlação fraca ou nula entre popularidade e frequência de releases ou atualizações.*

2. Contexto

Este é o primeiro laboratório da disciplina (Lab01) e não depende de entregas anteriores. Ele marca o início da mineração dos repositórios que será reaproveitada em laboratórios futuros, além de ser a primeira vez que o grupo estrutura seu processo de trabalho por meio do GitHub Projects (Kanban), prática que será mantida ao longo do semestre.

O objeto de estudo são os 1.000 repositórios com maior número de estrelas no GitHub, coletados em 19/08/2026 por meio da API GraphQL (api.github.com/graphql), sem o uso de bibliotecas de terceiros. Essa abordagem permite maior controle sobre as queries utilizadas e facilita a rastreabilidade dos dados coletados.

Para a RQ05 e a RQ07, que tratam de linguagens de programação, a referência de “popularidade” adotada é o relatório anual State of the Octoverse (octoverse.github.com), publicado pelo próprio GitHub. Dessa forma, mantém-se o mesmo critério de popularidade ao longo da análise, evitando inconsistências entre a referência utilizada para as linguagens e a fonte dos dados dos repositórios.

3. Metodologia

A metodologia adotada neste laboratório foi organizada em três frentes principais: coleta dos dados dos repositórios populares do GitHub, organização e preparação dos dados para análise e acompanhamento do desenvolvimento por meio do GitHub Projects. O processo foi estruturado de acordo com as etapas previstas no enunciado do laboratório, contemplando inicialmente a implementação da consulta GraphQL para obtenção das métricas, posteriormente a ampliação da coleta para os 1.000 repositórios por meio de paginação e, por fim, a preparação dos dados para análise e visualização.

3.1 Principais Desafios

Um dos principais desafios técnicos encontrados foi realizar a coleta de uma quantidade elevada de repositórios utilizando diretamente a API GraphQL do GitHub. Como o laboratório determina a utilização de uma consulta própria, sem bibliotecas de terceiros destinadas à consulta da API, foi necessário estruturar a requisição de forma que contemplasse as informações necessárias para responder às questões de pesquisa.

Outro desafio esteve relacionado à obtenção dos 1.000 repositórios. Uma única requisição não era suficiente para coletar todo o conjunto de dados necessário, tornando necessária a implementação de paginação. Dessa forma, a consulta foi adaptada para percorrer diferentes páginas de resultados até atingir a quantidade de repositórios definida pelo laboratório.

Também foi necessário organizar os diferentes dados retornados pela API em uma estrutura que pudesse ser utilizada nas etapas posteriores do trabalho. Para isso, os resultados foram tratados e exportados para arquivos CSV, permitindo que as informações coletadas fossem armazenadas de maneira estruturada e posteriormente utilizadas na análise das métricas.

No aspecto de processo, outro desafio foi organizar as atividades do grupo utilizando o GitHub Projects, uma vez que o laboratório determina que as tarefas sejam registradas como Issues reais, atribuídas a responsáveis e movimentadas pelas diferentes etapas do fluxo de trabalho. O próprio enunciado também determina que os commits sejam relacionados às Issues correspondentes, permitindo maior rastreabilidade entre as atividades planejadas e o desenvolvimento realizado.

3.2 Tomadas de Decisão

- **Limite de WIP da coluna Doing: WIP = 3:** foi definido um limite de três tarefas simultâneas na coluna Doing para evitar que muitas atividades fossem iniciadas ao mesmo tempo e permanecessem inacabadas. A escolha busca equilibrar a possibilidade de trabalhar em paralelo com a necessidade de concluir as tarefas antes de iniciar novas atividades. Como trade-off, um limite menor poderia reduzir ainda mais o trabalho em paralelo, mas também poderia deixar integrantes ociosos; um limite maior aumentaria a flexibilidade, porém dificultaria o controle do fluxo.
- **Definição operacional de “PR aceita” (RQ02):** foram consideradas aceitas apenas as pull requests com estado MERGED, utilizando `pullRequests(states: MERGED)`. Pull requests apenas fechadas, mas sem merge, não foram contabilizadas, pois podem ter sido rejeitadas ou abandonadas. A alternativa de utilizar CLOSED aumentaria a quantidade de contribuições consideradas, mas incluiria alterações que não foram incorporadas ao código. Optou-se, portanto, por MERGED para representar efetivamente as contribuições externas aceitas pelo projeto.
- **Tratamento de repositórios sem issues (RQ06):** quando um repositório não possui nenhuma issue, a razão entre issues fechadas e issues totais foi definida como 0. A alternativa seria representar o resultado como NaN ou excluir o repositório do cálculo. Optou-se por atribuir 0 para manter todos os repositórios da amostra e evitar que casos sem issues fossem descartados ou impedissem o cálculo da mediana.
- **Critério de inclusão da amostra:** foram mantidos todos os 1.000 repositórios retornados pela busca `stars:>1 sort:stars-desc`, sem aplicação de filtros adicionais. Repositórios sem linguagem primária, por exemplo, não foram excluídos e foram classificados como “Sem linguagem” nas RQ05 e RQ07. A alternativa de excluir esses repositórios simplificaria algumas análises, mas reduziria a amostra e poderia introduzir um viés ao eliminar determinados tipos de projetos. Assim, priorizou-se a manutenção dos 1.000 repositórios definidos pelo enunciado.

- **Paginação em duas fases (busca leve + detalhes em lote):** inicialmente, foi considerada a possibilidade de realizar uma única consulta contendo todos os campos necessários para os 1.000 repositórios. Essa abordagem, entretanto, apresentou problemas de timeout (HTTP 502 / 504) no resolver do GitHub devido ao volume de dados solicitado. Como alternativa, a coleta foi dividida em duas etapas: uma busca paginada com campos mais leves e, posteriormente, o enriquecimento dos dados com campos mais pesados em lotes menores, utilizando novas tentativas em caso de falhas temporárias. Essa abordagem aumenta a complexidade da implementação, mas tornou a coleta mais estável e viável.
- **Uso da mediana como medida central:** foi adotada a mediana para as métricas numéricas das RQ01–RQ04 e RQ07. A média foi considerada menos adequada devido à distribuição assimétrica de métricas como estrelas, releases e pull requests, na qual poucos repositórios podem apresentar valores muito elevados. O uso da mediana reduz a influência desses valores extremos e representa melhor o comportamento de um repositório típico da amostra. Como trade-off, a mediana não considera a magnitude dos valores extremos da mesma forma que a média, mas oferece maior robustez para a distribuição observada.

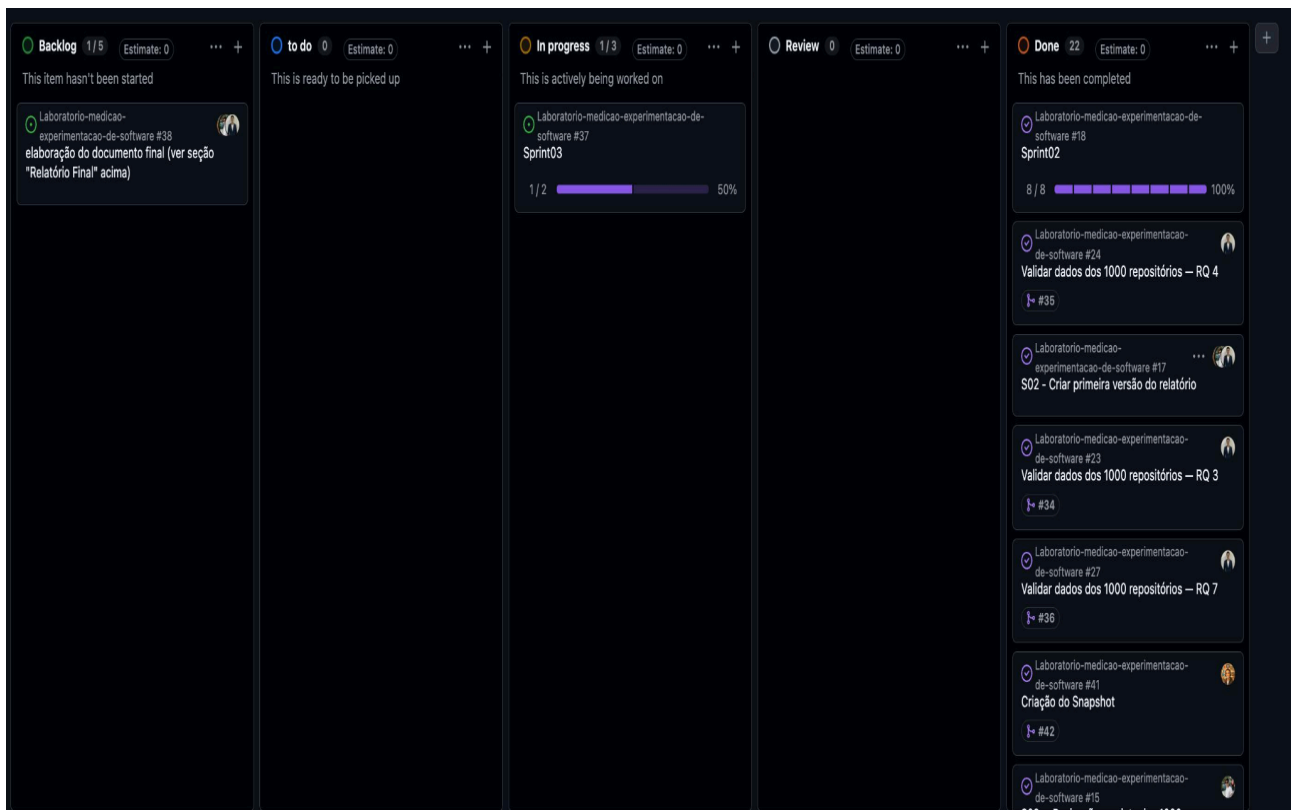
3.3 Etapas

O desenvolvimento do laboratório foi organizado em sprints, conforme a estrutura definida no enunciado. A primeira etapa, Lab01S01, concentrou-se na implementação da consulta GraphQL para obtenção das informações necessárias às questões de pesquisa, na realização das primeiras validações e na criação do GitHub Projects do grupo.

Na segunda etapa, Lab01S02, o foco passou para a ampliação da coleta por meio da implementação da paginação, permitindo alcançar os 1.000 repositórios previstos. Nessa etapa também foi realizada a organização dos dados em arquivos CSV e iniciada a preparação da base para as análises posteriores. O board do GitHub Projects também foi atualizado para refletir o andamento real das atividades.

Na terceira etapa, Lab01S03, foram realizadas as atividades relacionadas à análise e visualização dos dados coletados para as sete questões de pesquisa. A partir da base consolidada, os dados puderam ser utilizados para calcular e comparar as métricas definidas no laboratório.

Por fim, foi realizada a consolidação dos resultados e a elaboração do relatório final, reunindo a metodologia utilizada, os resultados obtidos, a discussão das hipóteses e a configuração do processo de desenvolvimento.



3.4 Ferramentas

As principais ferramentas e tecnologias utilizadas na coleta, processamento e análise dos dados foram:

- **Visual Studio Code (VS Code):** ambiente de desenvolvimento utilizado para implementação e execução dos scripts do laboratório.
- **GitHub GraphQL API:** utilizada para a coleta dos dados dos repositórios. As consultas foram desenvolvidas pelo próprio grupo, sem o uso de bibliotecas de terceiros destinadas à consulta da API, conforme exigido pelo laboratório.
- **Python:** utilizada no desenvolvimento dos scripts de coleta, processamento e organização dos dados.
- **Pandas:** utilizada para manipulação, tratamento e análise dos dados coletados.
- **CSV:** utilizado como formato estruturado para armazenamento dos dados coletados e dos resultados intermediários.
- **GitHub:** utilizado como fonte dos repositórios analisados e das informações obtidas por meio da API.
- **GitHub Projects (v2):** utilizado para o gerenciamento das Issues e acompanhamento do fluxo de desenvolvimento do grupo durante o laboratório. **Link:** <https://github.com/users/Enzocasaes/projects/2/views/1>

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Diferença entre a data de referência da coleta ([DATA]) e o campo createdAt, convertida em dias (arredondada para baixo).	Dias	GitHub GraphQL API
RQ02	Pull requests aceitas	Contagem de pull requests com estado MERGED.	Quantidade	GitHub GraphQL API
RQ03	Releases	Contagem total de releases do repositório.	Quantidade	GitHub GraphQL API
RQ04	Tempo até a última atualização	Diferença entre a data de referência da coleta e o campo updatedAt, convertida em dias.	Dias	GitHub GraphQL API
RQ05	Linguagem primária	primáriaValor do campo primaryLanguage.name; repositórios sem linguagem definida são categorizados como "Não especificado".	Categoria	GitHub GraphQL API
RQ06	Percentual de issues fechadas	issues fechadas(issues(states: CLOSED).totalCount dividido por issues.totalCount) multiplicado por 100, arredondado a 2 casas decimais; repositórios com 0 issues excluídos (N/A).	Percentual (%)	GitHub GraphQL API
RQ07	Contribuição, releases e atualização por linguagem	Para cada linguagem primária (RQ05), calcula-se a mediana das métricas RQ02, RQ03 e RQ04 entre os repositórios do grupo; linguagens com menos de 5 repositórios excluídas da comparação.	Quantidade, quantidade e dias	GitHub GraphQL API
RQ08	Correlação entre popularidade e as demais métricas	Coefficiente de correlação de Pearson entre stargazerCount (estrelas) e cada uma de: pull requests aceitas (RQ02), releases (RQ03), dias desde a última atualização (RQ04)	Coefficiente r (-1 a 1)	GitHub GraphQL API

3.6 Inovações Propostas pelo Grupo (30% da nota)

1. Análise de distribuição e detecção de outliers pelo método do IQR (RQ03 e RQ04). O enunciado solicita apenas os valores mínimo, máximo e mediana por RQ. Para as RQ03 e RQ04, o grupo implementou uma análise adicional em `estatisticas.js`, por meio das funções `calcularQuartis` e `detectarOutliers`, que calculam o primeiro e o terceiro quartis, o intervalo interquartil (IQR) e identificam os repositórios que estão fora da faixa $Q1 - 1,5 \times IQR$ a $Q3 + 1,5 \times IQR$,

seguindo o critério tradicional utilizado em boxplots. Essas funções são executadas pelos scripts de validação (`npm run validar:rq03` e `npm run validar:rq04`), que apresentam os outliers como informações sobre a distribuição dos dados, e não como falhas de coleta.

A motivação para essa análise é que os valores mínimo e máximo, quando analisados isoladamente, não permitem compreender adequadamente a distribuição dos dados. Na RQ03, por exemplo, 27,6% dos repositórios não possuem nenhuma release, informação que não é evidenciada apenas pelas três estatísticas exigidas pelo enunciado. A inclusão dos quartis, do IQR e da proporção de outliers permite evidenciar melhor essa assimetria. Esses resultados são utilizados na discussão da seção 4.3 para comparar as hipóteses formuladas com os dados observados.

2. Identificação de censura à direita na coleta da RQ03. Durante a validação dos dados, o grupo identificou quatro repositórios que retornavam exatamente 1.000 releases, um valor que levantou a possibilidade de existir um limite na consulta realizada. A verificação por meio da API REST, com paginação dos releases de cada repositório, mostrou que as contagens reais eram superiores a esse valor: 1.340 para `langchain-ai/langchain`, 1.991 para `electron/electron`, 3.824 para `vercel/next.js` e 6.958 para `ggml-org/llama.cpp`.

A partir dessa verificação, concluiu-se que o campo `releases.totalCount` da API GraphQL possui saturação em 1.000, fazendo com que valores superiores a esse limite sejam representados como 1.000. Dessa forma, os dados da RQ03 apresentam censura à direita. Como consequência, estatísticas sensíveis ao extremo superior, como o máximo, a média e o limite superior utilizado na identificação de outliers, devem ser interpretadas como limites inferiores dos valores reais. A mediana e os quartis inferiores, por estarem muito abaixo do ponto de saturação, não são afetados por essa limitação. O grupo optou por manter os dados obtidos na coleta original e registrar a limitação, em vez de realizar uma segunda coleta individual para os 1.000 repositórios, o que aumentaria significativamente a complexidade da coleta e reduziria a padronização do procedimento.

3. Painel visual estático e autocontido (`dashboard.js`). Além dos arquivos CSV previstos no enunciado, o grupo implementou um gerador de painel por meio do script `dashboard.js`. O script lê os arquivos já coletados no diretório `data/` e gera um único arquivo `dashboard.html`, com os dados incorporados no próprio documento. O painel não depende de um back-end ou de uma nova execução da coleta e pode ser aberto diretamente no navegador.

A proposta tem como objetivo facilitar a exploração dos resultados pelo grupo e pelo professor, permitindo visualizar os dados sem a necessidade de tokens, acesso à API ou conexão com o ambiente de coleta. Dessa forma, o painel complementa os resultados apresentados no relatório e facilita a consulta aos dados já obtidos.

4. Nova Questão de Pesquisa. Correlação entre popularidade e as demais métricas (RQ08). As questões RQ01 a RQ07 do enunciado utilizam o número de estrelas do repositório apenas como critério de amostragem — nenhuma delas avalia se a popularidade de fato se relaciona com as demais características medidas. O grupo propôs uma oitava questão de pesquisa, RQ08: existe correlação entre a popularidade de um repositório (número de estrelas) e as métricas de contribuição externa, frequência

de releases e frequência de atualização (RQ02, RQ03 e RQ04, respectivamente)? Para viabilizar essa análise, o campo `stargazerCount` foi incluído na consulta GraphQL já existente (`busca-populares.js`), e uma nova função genérica, `correlacaoPearson`, foi implementada em `estatisticas.js`, calculando o coeficiente de correlação de Pearson entre duas variáveis numéricas. A função foi validada com três casos de referência — correlação perfeita positiva, perfeita negativa e nula — antes de ser aplicada aos dados reais. A extração dos dados e o cálculo das correlações foram implementados em `rqs/rq08.js`, e a validação estrutural correspondente em `validacoes/rq08.js` (`npm run test:rq08` e `npm run validar:rq08`), seguindo o mesmo padrão de checagens (cabeçalho, quantidade de linhas, duplicidade, campos vazios e faixa numérica válida) já utilizado nas demais RQs.

A motivação para essa questão decorre da própria pergunta que orienta o laboratório: se a popularidade de um repositório é um indicador confiável de suas características de engenharia, ou se essas dimensões são independentes. As RQ01 a RQ07 respondem a isso de forma indireta, descrevendo cada característica isoladamente; a RQ08 testa essa relação diretamente, por meio de uma medida estatística de associação entre as variáveis. Como o coeficiente de Pearson mede apenas relações lineares, foi adotada uma escala fixa de interpretação ($|r| < 0,1$: nula; $0,1-0,3$: fraca; $0,3-0,5$: moderada; $> 0,5$: forte), evitando uma leitura subjetiva dos resultados. Vale notar, ainda, que a saturação do campo `releases.totalCount` em 1.000 (item 2 desta seção) pode atenuar levemente a correlação observada entre estrelas e releases, já que repositórios com contagens reais muito superiores ao limite são registrados com o mesmo valor máximo. Os coeficientes obtidos para os 1.000 repositórios coletados, bem como sua interpretação em relação à hipótese formulada, são apresentados na seção 4.3.

4. Resultados

4.1 Coleta de Dados

A coleta dos dados foi realizada utilizando a API GraphQL do GitHub, por meio de um script próprio desenvolvido pelo grupo. O processo foi estruturado para buscar os repositórios mais populares da plataforma, ordenados de forma decrescente pelo número de estrelas, e posteriormente obter as informações necessárias para responder às sete questões de pesquisa.

Na versão final da implementação, a quantidade de repositórios foi definida como 1.000, correspondendo ao conjunto estabelecido no enunciado do laboratório. Para alcançar esse volume, foi implementado um mecanismo de paginação baseado nos campos `hasNextPage` e `endCursor` retornados pela API. A implementação também utiliza lotes de requisições para obter os detalhes dos repositórios, evitando concentrar uma quantidade excessiva de informações em uma única requisição.

A coleta foi realizada utilizando uma única base de repositórios, sobre a qual foram calculadas as métricas das RQs01 a RQ08. Dessa forma, as diferentes questões de pesquisa utilizam o mesmo conjunto de dados, evitando diferenças na composição da amostra entre as análises. O projeto também registra que a coleta oficial dos 1.000 repositórios foi executada com sucesso e que foram gerados os arquivos CSV correspondentes às sete RQs.

Para verificar a consistência dos dados, foram implementadas validações estruturais dos arquivos gerados. Entre as verificações realizadas estão a conferência do cabeçalho, quantidade esperada de registros, ausência de repositórios duplicados e ausência de campos vazios. Também foram implementadas verificações específicas para as métricas, como a consistência entre o número de issues fechadas e o total de issues e a correspondência entre a razão calculada e as contagens utilizadas.

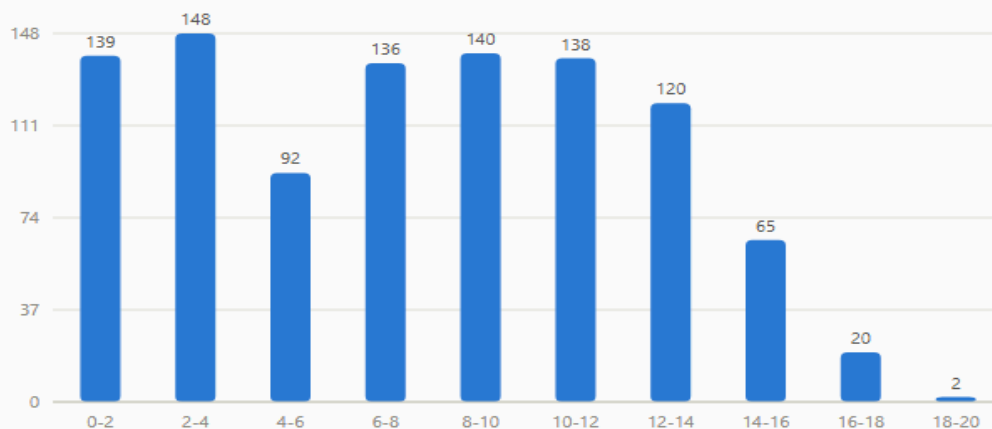
A implementação utiliza a mediana como principal medida de tendência central para as métricas numéricas. Essa escolha é especialmente relevante para dados de repositórios de software, nos quais alguns projetos podem apresentar valores muito superiores aos demais, principalmente em métricas como pull requests, releases e quantidade de issues.

4.2 Visualização Gráfica

RQ01

Sistemas populares são maduros/antigos?

Distribuição da idade dos 1.000 repositórios (anos desde a criação), em faixas de 2 anos.

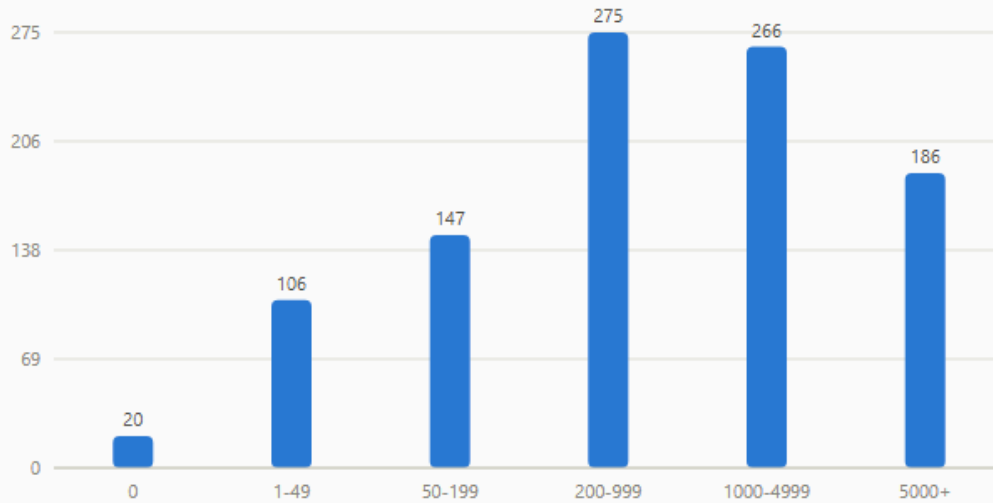


Mediana: **7,74 anos** · mínimo 0,02 · máximo 18,36.

RQ02

Sistemas populares recebem muita contribuição externa?

Distribuição de pull requests aceitas (MERGED), em faixas.

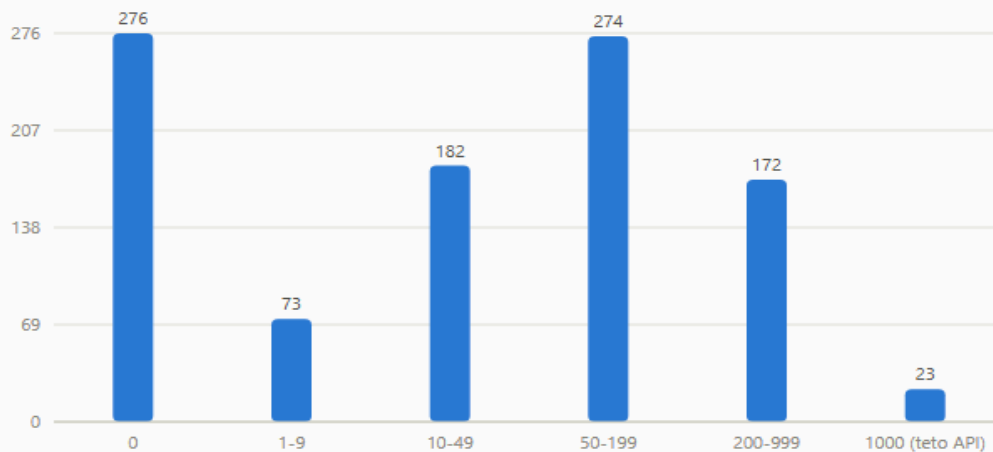


Mediana: **768 PRs aceitas** · apenas 2% dos repositórios sem nenhuma PR aceita.

RQ03

Sistemas populares lançam releases com frequência?

Distribuição do total de releases, em faixas.

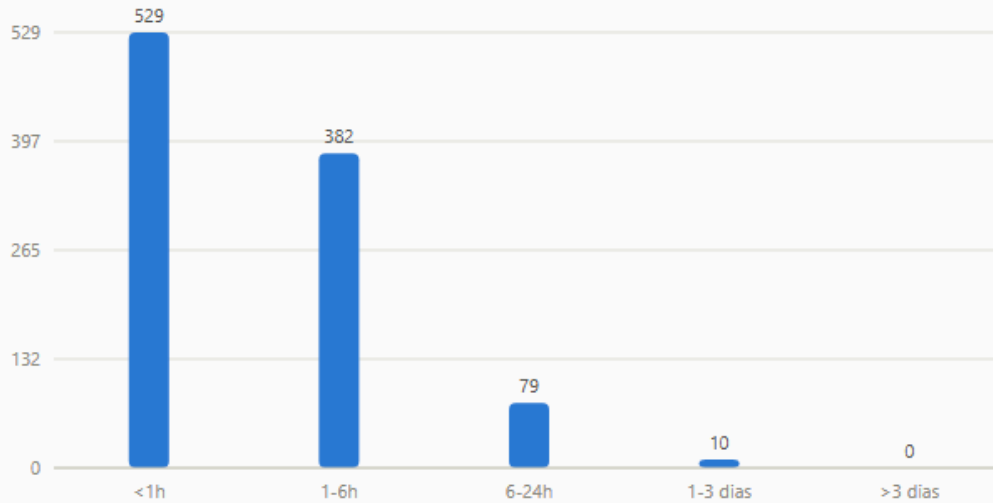


Mediana: **41 releases** · 27,6% dos repositórios sem nenhuma release · a faixa "1000" reúne repositórios no teto aparente da API (não necessariamente o valor real).

RQ04

Sistemas populares são atualizados com frequência?

Distribuição do tempo desde a última atualização, em faixas de horas/dias.

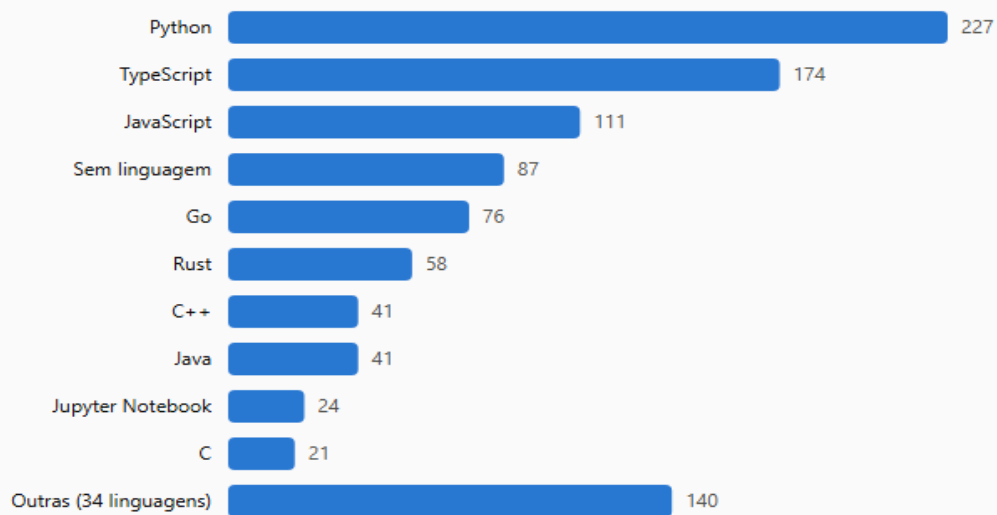


Mediana: **0,04 dia (~1h)** · 100% dos repositórios atualizados na última semana.

RQ05

Sistemas populares são escritos nas linguagens mais populares?

Top 10 linguagens primárias (de 44 distintas) por número de repositórios.

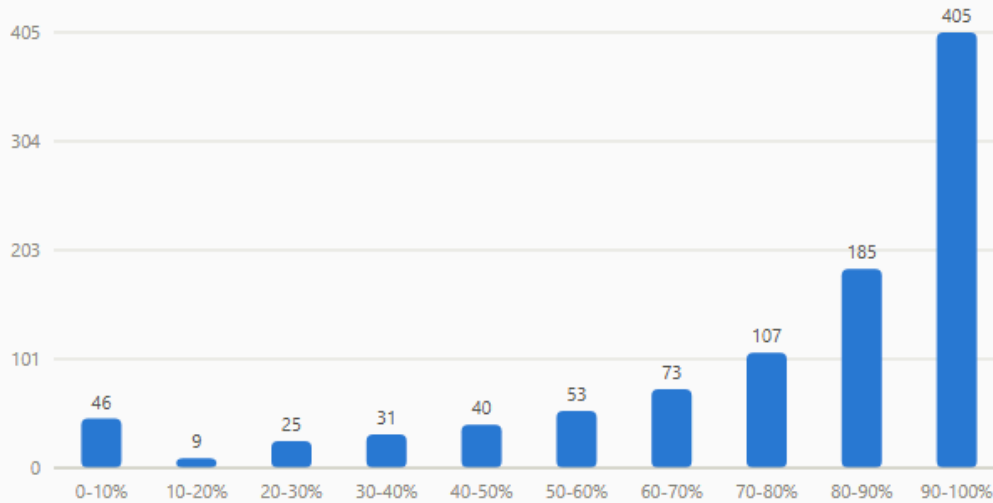


Fonte de referência para "linguagens mais populares": GitHub Octoverse. Python, TypeScript e JavaScript concentram 51,2% da amostra.

RQ06

Sistemas populares possuem um alto percentual de issues fechadas?

Distribuição da razão issues fechadas / total de issues, em faixas de 10%.

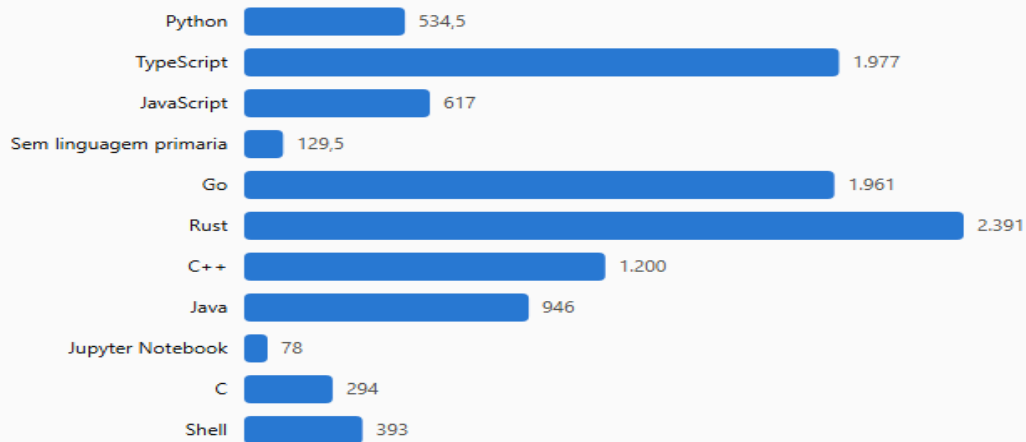


Mediana: **86%** - concentração forte acima de 80%.

RQ07 (1/3)

Sistemas em linguagens populares recebem mais contribuição externa?

Mediana de pull requests aceitas por linguagem (grupos com ≥ 20 repositórios).

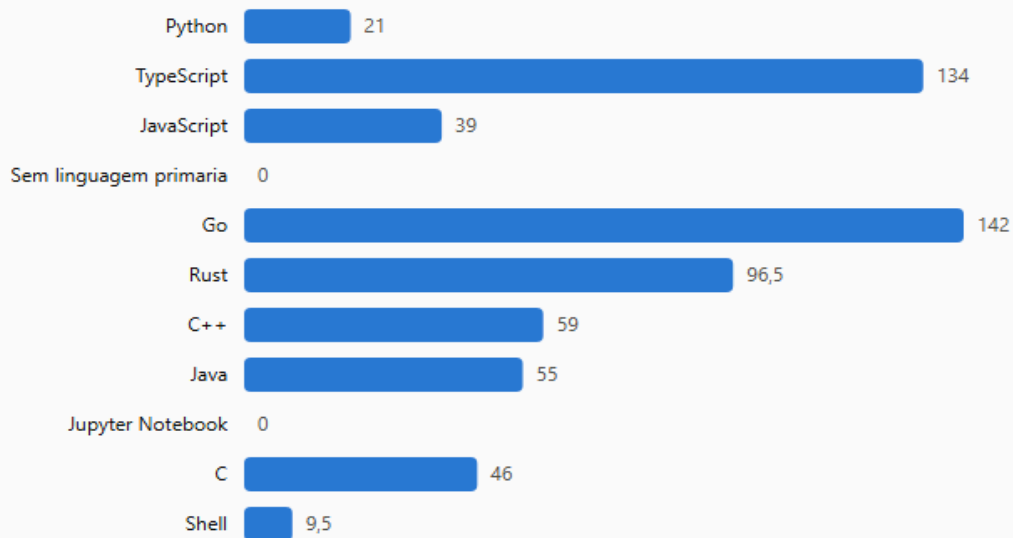


Rust, TypeScript e Go têm medianas de PRs aceitas acima de Python; Jupyter Notebook é o menor grupo grande (78 PRs).

RQ07 (2/3)

Sistemas em linguagens populares lançam mais releases?

Mediana de releases por linguagem (grupos com ≥ 20 repositórios).

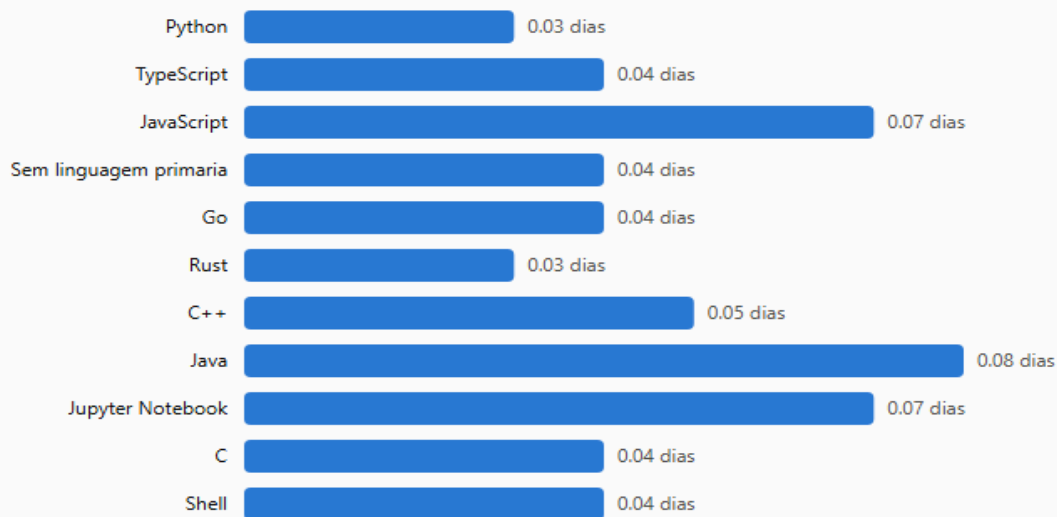


Go e TypeScript lideram; Jupyter Notebook e "Sem linguagem primária" têm mediana zero.

RQ07 (3/3)

Sistemas em linguagens populares são atualizados com mais frequência?

Mediana de dias desde a última atualização por linguagem (grupos com ≥ 20 repositórios).

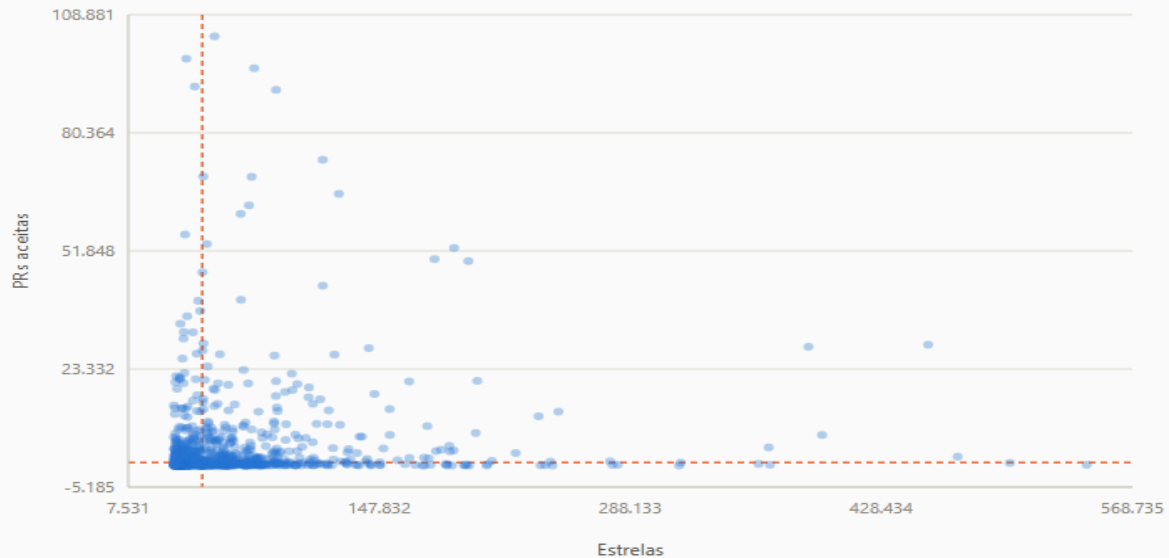


Todas as linguagens ficam entre 0,03 e 0,08 dia — a linguagem **não** diferencia a frequência de atualização, ao contrário do esperado pela hipótese.

RQ08 (1/3)

Existe correlação entre popularidade (estrelas) e contribuição externa (PRs aceitas)?

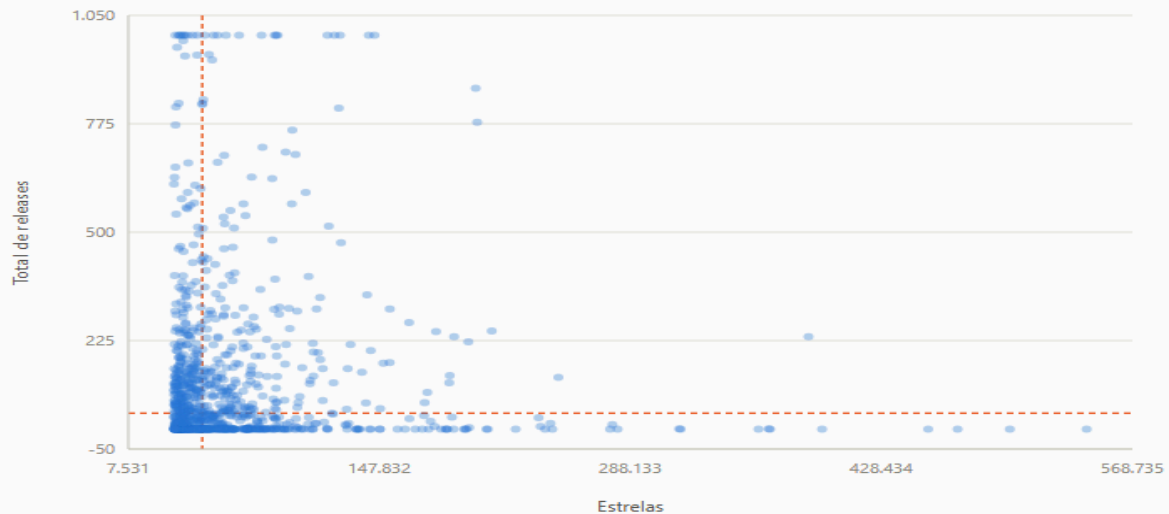
Dispersão: cada ponto é 1 dos 1.000 repositórios, estrelas (eixo X) × PRs aceitas (eixo Y).



RQ08 (2/3)

Existe correlação entre popularidade (estrelas) e frequência de releases?

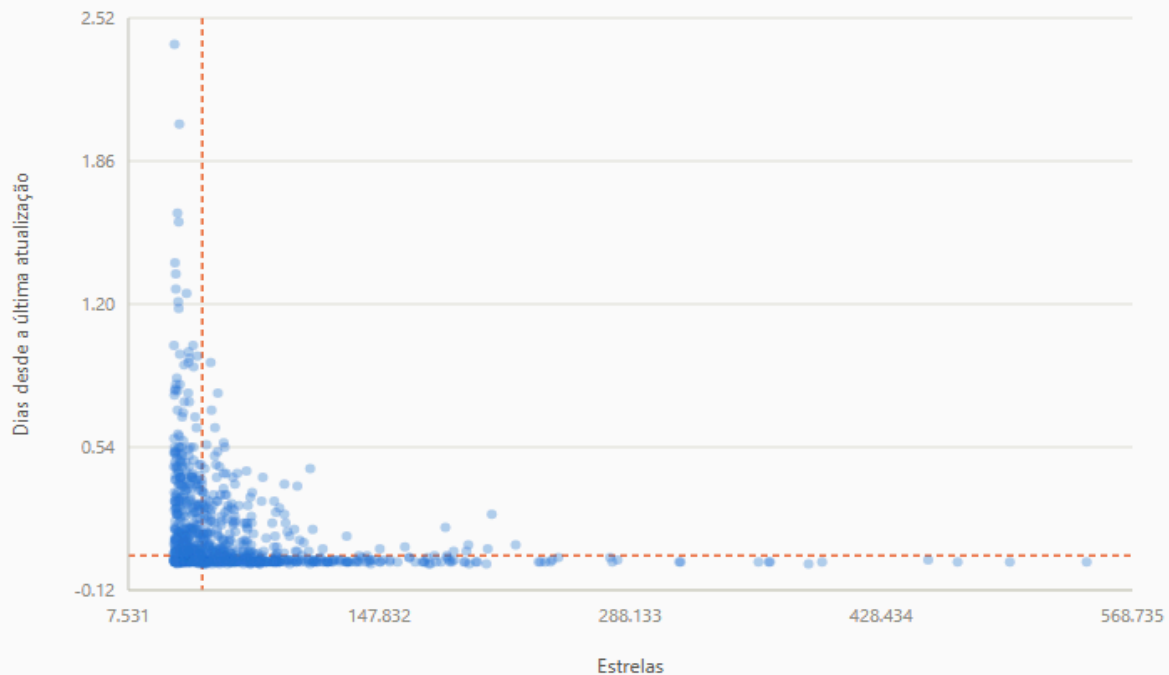
Dispersão: cada ponto é 1 dos 1.000 repositórios, estrelas (eixo X) × total de releases (eixo Y).



RQ08 (3/3)

Existe correlação entre popularidade (estrelas) e frequência de atualização?

Dispersão: cada ponto é 1 dos 1.000 repositórios, estrelas (eixo X) × dias desde a última atualização (eixo Y).



Mediana: **48.878 estrelas** / **0.04 dias**. Correlação de Pearson $r = -0.207$ — fraca (a única das três não-desprezível): repositórios com mais estrelas tendem a estar marginalmente mais atualizados, mas o efeito é pequeno.

● cada ponto = 1 repositório (1000 no total, com transparência p/ sobreposição) · --- linha tracejada = mediana de cada eixo

4.3 Discussão

(Comparação hipótese vs. resultado, RQ por RQ, com os números reais coletados hoje.)

RQ01 — maturidade: hipótese era que os repositórios populares seriam, em sua maioria, maduros. Confirmada: mediana de 7,74 anos de idade, com máximo de 18,36 anos (repositórios praticamente da fundação do GitHub, 2008). Poucos repositórios muito jovens (mínimo de 0,02 ano $\approx$ 1 semana) mostram que popularidade recente também é possível, mas é exceção, não regra.

RQ02 — contribuição externa: hipótese de número elevado de PRs aceitas. Confirmada: mediana de 768 PRs aceitas, e apenas 2% dos repositórios sem nenhuma PR aceita — a esmagadora maioria dos projetos populares tem um histórico real de contribuição externa.

RQ03 — frequência de releases: hipótese de que projetos populares publicam releases "com relativa frequência". Parcialmente confirmada. A mediana (41 releases) sugere prática de versionamento comum, mas 27,6% dos repositórios não têm nenhuma release — quase 1/3 do

total. Isso mostra que "popular" não implica "segue um processo formal de versionamento": listas `awesome-*`, `dotfiles` e tutoriais, por exemplo, raramente lançam releases, mesmo sendo muito populares. A hipótese inicial, ao falar só em "frequência", não previu essa divisão bimodal da amostra.

RQ04 — frequência de atualização: hipótese de atualização frequente. Confirmada, e de forma mais extrema que o esperado: mediana de 0,04 dia (cerca de 1 hora) e 100% dos 1.000 repositórios atualizados na última semana. Ressalva importante para a validade: `updatedAt` muda com qualquer push (bots de dependência, CI, traduções automatizadas), então esse resultado mede *atividade no repositório*, não necessariamente desenvolvimento humano ativo — um repositório pode estar "tecnicamente" atualizado só por commits automatizados.

RQ05 — linguagens populares: hipótese de que as linguagens predominantes estariam entre as mais usadas segundo o Octoverse. Confirmada: Python (22,7%), TypeScript (17,4%) e JavaScript (11,1%) lideram — todas entre as linguagens mais citadas nos relatórios anuais do Octoverse nos últimos anos.

RQ06 — issues fechadas: hipótese de proporção elevada. Confirmada: mediana de 86% de issues fechadas, indicando manutenção ativa na maioria dos projetos populares.

RQ07 — combinação por linguagem: hipótese de que linguagens populares teriam mais contribuição externa, mais releases e atualização mais frequente. Parcialmente confirmada. Nos grupos com amostra razoável (≥ 24 repositórios), TypeScript, Go e Rust têm medianas de PRs aceitas e releases visivelmente acima de linguagens de conteúdo/dados como Jupyter Notebook — essa parte da hipótese se confirma. Porém, a dimensão "atualizações mais frequentes" não se confirma como diferenciador: a mediana de dias-desde-atualização fica muito baixa (entre 0,01 e 0,3 dia) em praticamente todas as linguagens, então esse aspecto da RQ07 não distingue bem projetos por linguagem — todo repositório popular está "sempre atualizado", independente da linguagem. Ressalva adicional: 24 das 44 linguagens têm menos de 5 repositórios na amostra, então médias/medianas isoladas desses grupos (ex.: LLVM aparecendo como "maior mediana de PRs" com um único repositório) não representam uma tendência real da linguagem, e não devem ser lidas como conclusão.

RQ08 — correlação entre estrelas e as demais métricas (inovação): hipótese de correlação positiva fraca a moderada entre estrelas e PRs aceitas, e correlação fraca ou nula entre estrelas e releases/atualização. Hipótese refutada na parte principal. O coeficiente de Pearson entre estrelas e PRs aceitas é $r = 0,084$ — praticamente nulo, não fraco-positivo como esperado: dentro do top-1000 por estrelas, ter ainda mais estrelas não anda junto com receber mais contribuição externa. A correlação entre estrelas e releases também é nula ($r = -0,025$) — resultado que deve ser lido com a ressalva de que o campo `releases.totalCount` está sujeito à saturação em 1.000 descrita na seção 3.6 (item 2), o que pode atenuar levemente essa correlação para os repositórios com contagens reais mais altas. A única correlação não-desprezível é entre estrelas e dias desde a última atualização ($r = -0,209$, fraca): mais estrelas associam-se a uma atualização ligeiramente mais recente, mas o efeito é pequeno. Esse é o achado mais importante do laboratório: dentro da amostra já filtrada para "muito

popular", a popularidade deixa de diferenciar contribuição externa ou disciplina de releases — reforça, com dado direto (e não apenas pela comparação indireta entre as demais RQs), a pergunta motivadora da Introdução. Ressalva de interpretação: o coeficiente de Pearson mede associação linear, não causalidade — não é possível concluir que "mais estrelas causam menos PRs", apenas que, nesta amostra, essas variáveis não andam juntas.

Ameaças à validade:

- Amostra de um único instante: a coleta é uma fotografia de um momento específico (26/08/2026); popularidade e atividade mudam ao longo do tempo, então os números não são estáveis no médio prazo.
- Limite aparente da API em `releases.totalCount`: dois repositórios cravados em exatamente 1000 releases sugerem um teto artificial da API, não o valor real — pode subestimar outliers reais acima desse teto.
- `updatedAt` como proxy de atividade: conforme discutido na RQ04, mede push (inclusive automatizado), não necessariamente desenvolvimento humano — enfraquece a RQ04 e a dimensão de atualização da RQ07 como indicador de "atividade real".
- Grupos pequenos na RQ07: mais da metade das linguagens (24/44) tem menos de 5 repositórios, tornando comparações entre linguagens de nicho pouco confiáveis.

O que as inovações do grupo (seção 3.6) acrescentaram: o relatório de distribuição/outliers foi o que permitiu identificar a divisão bimodal da RQ03 (27,6% sem release) e o teto suspeito de 1000 releases na API — sem ele, a mediana isolada esconderia os dois achados.

5. Conclusão

Este laboratório investigou se a popularidade de um repositório no GitHub (medida por estrelas) se relaciona com maturidade, contribuição externa, disciplina de versionamento, frequência de atualização, escolha de linguagem e gestão de issues. Das sete questões de pesquisa, cinco tiveram hipótese confirmada de forma direta (RQ01 maturidade, RQ02 contribuição externa, RQ04 atualização, RQ05 linguagens, RQ06 issues fechadas) e duas foram apenas parcialmente confirmadas (RQ03 releases — quase 1/3 dos repositórios sem nenhuma release, apesar da mediana razoável; RQ07 — a relação com contribuição/releases se confirma por linguagem, mas a relação com frequência de atualização não, porque virtualmente todo repositório popular está "sempre atualizado" independente da linguagem).

A principal limitação do estudo é temporal: a coleta é um retrato único no tempo, sem acompanhamento longitudinal, então não é possível afirmar se essas características são estáveis ou apenas o estado atual desses projetos. Outra limitação é a dependência de campos calculados pela própria API do GitHub (ex.: `updatedAt`, `releases.totalCount`) sem controle direto sobre sua definição exata — o que levou aos dois pontos de ameaça à validade discutidos na seção 4.3 (teto de releases, `updatedAt` incluindo bots).

Com mais tempo, o grupo faria: (i) uma segunda coleta em outro momento do semestre, para verificar estabilidade dos resultados; (ii) uma investigação direta sobre o teto aparente de 1000 releases na API, consultando a documentação oficial do GitHub; (iii) expandir o relatório de distribuição/outliers (inovação da seção 3.6) para todas as sete RQs, não só RQ03/RQ04, o que provavelmente revelaria assimetrias parecidas em RQ02 (PRs) e RQ06 (issues).

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
- GitHub. GitHub GraphQL API documentation. Disponível em: <https://docs.github.com/graphql>. Acesso em: 26 ago. 2026.
- GitHub. Octoverse — State of the Octoverse. Disponível em: <https://octoverse.github.com>. Acesso em: 26 ago. 2026. *(fonte usada na RQ05/RQ07, já referenciada no corpo do texto — só falta constar aqui também)*